

4

Building a News App Using .NET MAUI Shell

In this chapter, we will create a news app that leverages the **Shell** navigation functionality provided to us by the .NET MAUI team at Microsoft. The old way of doing this, which involved using **ContentPage**, **FlyoutPage**, **TabbedPage**, or **NavigationPage** as the main page, as we did in *Chapter 2*, still works, but we are sure that you will enjoy the new way of defining the structure of your app. Also, you can mix and match the old and new.

By the end of this chapter, you will have learned how to define an app structure using Shell, consume data from a REST API, configure navigation, and pass data between views using query-style routes.

So, what is Shell, then? In Shell, you define the structure of your app using **Extensible Application Markup Language (XAML)** instead of hiding it in spread-out pieces of code in your app. You can also navigate using routes, just like those fancy web developers are doing.

The following topics will be covered in this chapter:

- Defining a Shell navigation page
- Creating a flyout
- Creating a navigation bar
- Navigating using routes and passing data in query strings
- Consuming data from a public **representational state transfer (REST) application programming interface (API)**
- Adding content in the form of a **CollectionView** control

Technical requirements

To be able to complete this project, you will need to have Visual Studio for Mac or Windows installed, as well as the necessary .NET MAUI workload components. See *Chapter 1, Introduction to .NET MAUI*, for more details on how to set up your environment.

You can find the source code for this chapter at

<https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition>.

Project overview

We will create a .NET MAUI project using the **single-project** feature as the code-sharing strategy. It will contain two parts, detailed as follows:

- In the first part, we will create views and make them navigable using Shell
- In the second part, we will add some content by consuming a **REST API** for news

The second part is not needed to learn about Shell, but it will take you a bit further down the road to a complete app.

The build time for this project is about 1.5 hours.

Building the news app

This chapter will be all about building a news app from the beginning. It will guide you through every step, but it will not go into every detail. For that, we recommend *Chapter 2, Building Our First .NET MAUI App*, which goes into more detail.

Happy coding!

Setting up the project

This project, like all the rest, is a **File | New | Project...**-style project. This means that we will not be importing any code at all. So, this first section is all about creating the project and setting up the basic project structure.

Creating the new project

The first step is to create a new .NET MAUI project:

1. Open Visual Studio 2022 and select **Create a new project**:

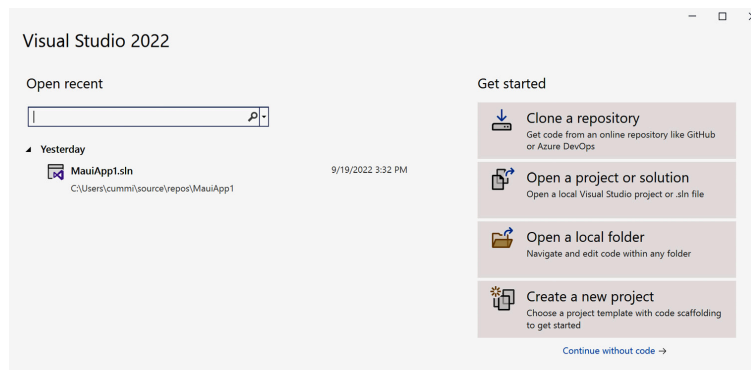


Figure 4.1 – Visual Studio 2022

This will open the **Create a new project** wizard.

2. In the search field, type in **maui** and select the **.NET MAUI App** item from the list:

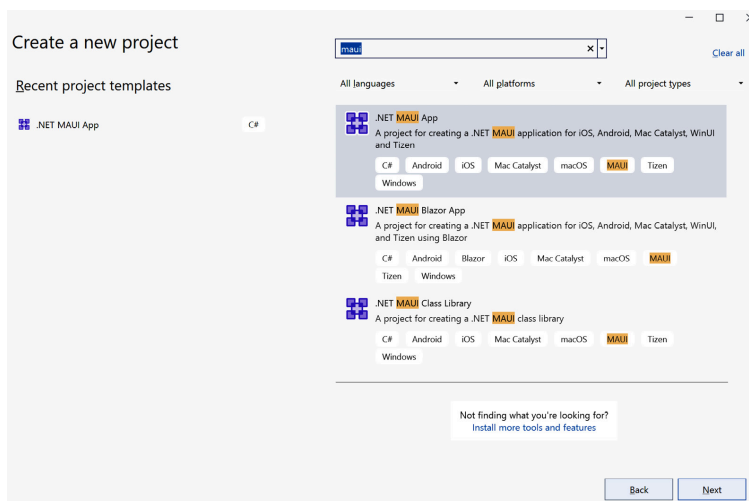


Figure 4.2 – Create a new project

3. Click **Next**.
4. Enter **News** as the name of the app, as shown in the following screenshot:

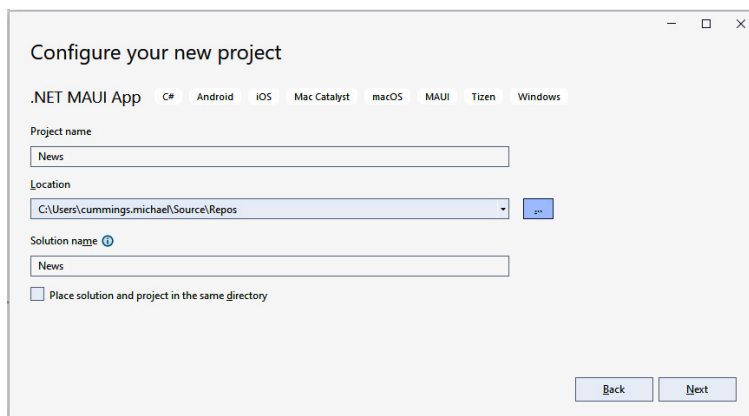


Figure 4.3 – Configure your new project

5. Click **Next**.
6. The last step will prompt you for the version of .NET Core to support. At the time of writing, .NET 6 is available as **Long-Term Support (LTS)**, and .NET 7 is available as **Standard Term Support**. For this book, we will assume that you will be using .NET 7:

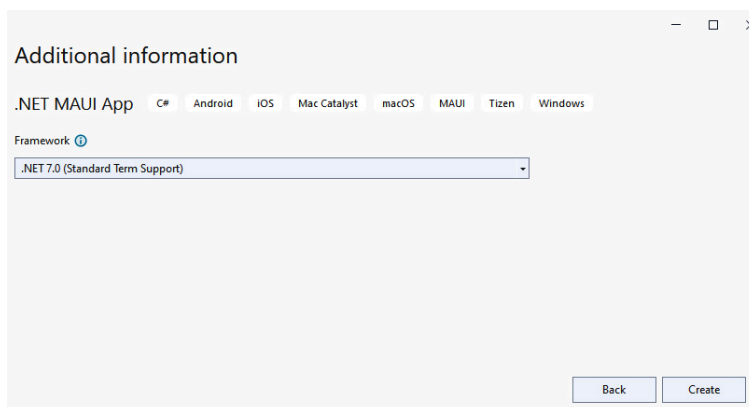


Figure 4.4 – Additional information

7. Finalize the setup by clicking **Create** and wait for Visual Studio to create the project.

That's it for project creation.

Let's continue by setting up the structure of the app.

Creating the structure of the app

In this section, we will start to build the **Views** and **ViewModels** of the app. The *Using MVVM – creating views and ViewModels* section in *Chapter 2* contains more details on **Model-View-ViewModel (MVVM)** as a design pattern. It's recommended that you read that first if you don't know what MVVM is.

Creating the ViewModel base class

ViewModel is the mediator between **View** and **Model**. Let's create a base class for **ViewModels** with common functionality that we can reuse. In practice, **ViewModel** must implement an interface called **INotifyPropertyChanged** for MVVM to function. We will do so in the base class and will also add a little handy helper tool called **CommunityToolkit.Mvvm** that will save us a lot of time. Again, please check out *Chapter 2, Building Our First .NET MAUI App*, if you are feeling unsure about MVVM.

The first step is to create a base class. Proceed as follows:

1. In the **News** project, create a folder called **ViewModels**.
2. In the **ViewModels** folder, create a class called **ViewModel**.
3. Change the existing class so that it looks as follows:

```
namespace News.ViewModels;
public abstract class ViewModel
{
}
```

Excellent! Let's implement **INotifyPropertyChanged** in the base **ViewModel** class.

A quick recap of CommunityToolkit.Mvvm

CommunityToolkit.Mvvm is a NuGet package that contains a few source generators that can automatically generate the necessary implementation details for **INotifyPropertyChanged**. More specifically, it will inject a call that will raise the **PropertyChanged** event whenever a setter is called. It also takes care of property dependencies; if I change the **FirstName** property, the **FullName** read-only property will also get a **PropertyChanged** event. Before CommunityToolkit.Mvvm, you would have had to write this code manually.

It's all explained in more detail in *Chapter 2, Building Our First .NET MAUI App*. Have you read it yet?

Adding a reference to `CommunityToolkit.Mvvm`

`CommunityToolkit.Mvvm` and its dependencies are installed using NuGet. So, let's install the NuGet package:

1. In the **News** project, install the `CommunityToolkit.Mvvm` NuGet package, version 8.0.0.
2. Accept any license dialog boxes.

This will install the relevant NuGet packages.

Implementing `INotifyPropertyChanged`

`ViewModel` sits between `View` and `Model`. When a change in `ViewModel` occurs, `View` must be notified. The mechanism for this is the `INotifyPropertyChanged` interface, which defines an event that the controls in `View` subscribe to. The `ObservableObject` attribute is the magic that will generate the `INotifyPropertyChanged` implementation for us. Follow these steps:

1. In the **News** project, open up `ViewModels.cs`.
2. Add the following code in bold:

```
using CommunityToolkit.Mvvm.ComponentModel;  
[ObservableObject]  
public abstract partial class ViewModel  
{  
}
```

This instructs `CommunityToolkit.Mvvm` to implement the `INotifyPropertyChanged` interface. The next step is all about reducing the number of lines of code that we will have to write. Normally, you would have to manually raise the `PropertyChanged` event from your code, but thanks to source generators, which write code at build time, we simply have to create normal properties and let `CommunityToolkit.Mvvm` do the magic.

Let's move on and create our first `ViewModel`.

Creating the `HeadlinesViewModel` class

We will now start to create some `View` and `ViewModel` placeholders that we will expand on during this chapter. We will not directly implement all graphical features; instead, we'll keep it simple and think of all these pages as placeholders for what's to come next.

The first one is the `HeadlinesViewModel` class, which will serve as the `ViewModel` for `HeadlinesView`. Proceed as follows:

1. In the **News** project, under the `ViewModels` folder, create a new class called `Head linesViewModel`.
2. Edit the class so that it inherits from the `ViewModel` base class, as shown in bold in the following code snippet:

```
namespace News.ViewModels;
public class HeadlinesViewModel : ViewModel
{
    public HeadlinesViewModel()
    {
    }
}
```

OK – not bad. It doesn't do much yet, but we'll just leave it for now. Let's create the matching view.

Creating HeadlinesView

This view will eventually show a list of news, but for now, it will be kept simple. Follow these steps to create the page:

1. In the **News** project, create a folder named **Views**.
2. Right-click on the **Views** folder, select **Add**, and then click **New Item....**
If you are using Visual Studio 17.7 or later, click the **Show all Templates** button in the dialog that pops up. Otherwise, move on to the next step.
3. Under the **C# Items** node on the left, select **.NET MAUI**.
4. Select **.NET MAUI ContentPage (XAML)** and name it **HeadlinesView**.
5. Click **Add** to create the page.

Refer to the following screenshot to view the preceding information:

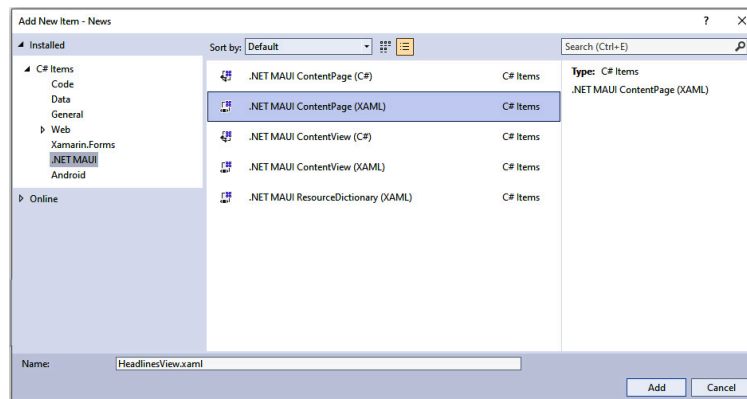


Figure 4.5 – Add New Item

Let's add some placeholder code to **HeadlinesView**, just to have something to navigate to and from. We will replace it with something hotter later on in this chapter, but to keep things simple, let's add a label. To do so, proceed as follows:

1. In the **News** project, under the **Views** folder, open **HeadlinesView.xaml**.
2. Edit the XAML code by adding the following code marked in bold:

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="News.Views.HeadlinesView"
              Title="Home">
    <VerticalStackLayout>
```

```

        <Label
            Text="HeadLinesView!"
            VerticalOptions="Center"
            HorizontalOptions="Center" />
    </VerticalStackLayout>
</ContentPage>

```

This will set the title of the page and add a label with the text **HeadlinesView** centered in the middle of the page. Let's move on and create some additional view placeholders.

Creating ArticleItem

The app will eventually display a list of articles where each article will be rendered using a reusable component. We will call this reusable component **ArticleItem**. In .NET MAUI, a reusable component is called a **ContentView**. Please don't confuse this with an MVVM View, which is represented by a page in .NET MAUI. We know that this is confusing, but the rule is that a .NET MAUI page is an MVVM View and a .NET MAUI ContentView is essentially a reusable control.

That said, let's create the **ArticleItem** class, as follows:

1. In the **News** project, right-click the **Views** folder, select **Add**, and then click **New Item....**
If you are using Visual Studio 17.7 or later, click the **Show all Templates** button in the dialog that pops up. Otherwise, move on to the next step.
2. Under the **C# Items** node on the left, select **.NET MAUI**.
Important: Make sure that you select the **ContentView** template in the next step and *not* the **ContentPage** template.
3. Select **.NET MAUI ContentView (XAML)** (important, remember?) and name it **ArticleItem**.
4. Click **Add** to create the view.

Refer to the following screenshot to view the preceding information:

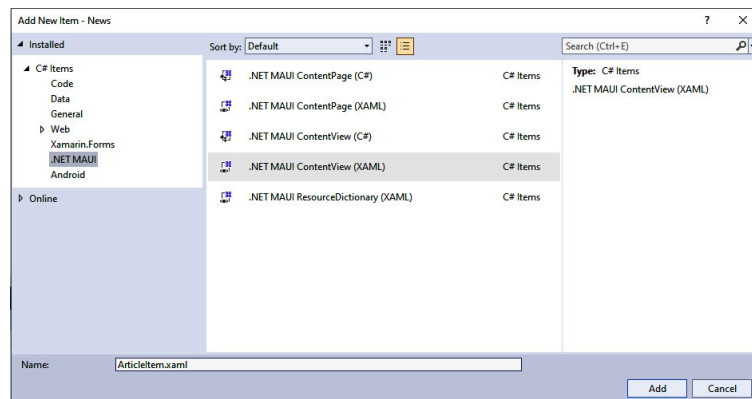


Figure 4.6 – Add New Item – ArticleItem.xaml

We don't need to alter the XAML code that's been generated at this point, so we'll simply leave it as is.

Creating ArticleView

In the previous section, we created the **ArticleItem** content view. This view (**ArticleView**) will contain **WebView** to display each article. But for the moment, let's just add **ArticleView** as a placeholder. Follow these steps to do so:

1. In the **News** project, right-click the **Views** folder, select **Add**, and then click **New Item....**
If you are using Visual Studio 17.7 or later, click the **Show all Templates** button in the dialog that pops up. Otherwise, move on to the next step.
2. Select **.NET MAUI** under the **C# Items** node on the left.
3. Select **.NET MAUI ContentPage (XAML)** and name it **ArticleView**.
4. Click **Add** to create the page.

Since this view is also a placeholder view at the moment, we'll just add a label to indicate the type of page. Edit the content by following these steps:

1. In the **News** project, open **ArticleView.xaml**.
2. Edit the XAML code by adding the following code marked in bold:

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  x:Class="News.Views.ArticleView"
  Title="ArticleView">
  <VerticalStackLayout>
    <Label
      Text="ArticleView!"
      VerticalOptions="Center"
      HorizontalOptions="Center" />
  </VerticalStackLayout>
</ContentPage>
```

Alright – just one more view to mock before we can start to wire things up.

Creating AboutView

The last view will be created in the same way as all the others. Proceed as follows:

1. In the **News** project, right-click the **Views** folder, select **Add**, and then click **New Item....**
2. Under the **C# Items** node on the left, select **.NET MAUI**.
3. Select **.NET MAUI ContentPage (XAML)** and name it **AboutView**.
4. Click **Add** to create the page.

This view is the only view that will stay a placeholder view. It's up to you to do something with it if you choose to build something cool out of this project later. So, we will only add a label that states that this is **AboutView**:

1. In the **News** project, open **AboutView.xaml**.
2. Edit the XAML code by adding the following code marked in bold:


```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="News.Views.AboutView"
             Title="AboutView">
  <VerticalStackLayout>
    <Label
      Text="AboutView!"
      VerticalOptions="Center"
      HorizontalOptions="Center" />
  </VerticalStackLayout>
</ContentPage>
```

With that, we have all the views we need to start wiring the app up. The first step is to configure dependency injection.

Wiring up dependency injection

By using dependency injection as a pattern, we can keep our code cleaner and more testable. This app will use constructor injection, which means that all dependencies that a class has must be passed through its constructor. The container then constructs objects for you, so you don't have to care too much about the dependency chain. Since .NET MAUI already includes a dependency injection framework, called `Microsoft.Extensions.DependencyInjection`, there is nothing extra to install.

CONFUSED ABOUT DEPENDENCY INJECTION?

Check out the Wiring up a dependency injection section in Chapter 2, Building Our First .NET MAUI App, for more details on dependency injection.

Registering Views and ViewModels with dependency injection

When registering classes with the container, it is recommended to use extension methods to group the types. The extension methods will take a single parameter and return a single value, the `MauiAppBuilder` instance. This is how the **Builder** pattern is implemented and it allows us to chain the methods on the builder defined in the `CreateMauiApp` method. For this app, we have **Views** and **ViewModels** to register for now. Let's create the method:

1. In the **News** project, open the `MauiProgram.cs` file.
2. Make the following changes to the `MauiProgram` class; the changes are highlighted in the code:

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
```

```

        .UseMauiApp<App>()
        .ConfigureFonts(fonts =>
        {
            fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
            fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
        })
        .RegisterAppTypes();
        return builder.Build();
    }
    public static MauiAppBuilder RegisterAppTypes(this MauiAppBuilder mauiAppBuilder)
    {
        // ViewModels
        mauiAppBuilder.Services.AddTransient<ViewModels.
        HeadlinesViewModel>();
        // Views
        mauiAppBuilder.Services.AddTransient<Views.AboutView>();
        mauiAppBuilder.Services.AddTransient<Views.
        ArticleView>();
        mauiAppBuilder.Services.AddTransient<Views.
        HeadlinesView>();
        return mauiAppBuilder;
    }
}

```

The .NET MAUI **MauiAppBuilder** class exposes the **Services** property, which is the dependency injection container. We simply need to add the types we want dependency injection to know about; the container will do the rest for us. Think of a builder as something that collects a lot of information on what needs to be done, and then finally builds the object we need. It's a very useful pattern on its own, by the way.

We're only using the builder for one thing at the moment. Later on, we will use it to register any class in the assembly that inherits from our abstract **ViewModel** class. The container is now prepared for us to ask for these types.

Now, we need to make some graphical touches to our app. We will rely on **Font Awesome** to do the magic.

Downloading and configuring Font Awesome

Font Awesome is a free collection of images packaged into a font. .NET MAUI has excellent support for using Font Awesome in toolbars, navigation bars, and all over the place. It's not strictly needed to make this app, but we think that it's worth the extra round trip since you are most likely going to need something like this in your new killer app.

The first step is to download the font.

Downloading Font Awesome

Downloading the font is straightforward. Please note the renaming of the file – it's not needed but it's easier to edit configuration files and such if they have simpler names. Follow these steps to acquire and copy the font to each project:

1. Browse to <https://fontawesome.com/download>.
2. Click the **Free for Desktop** button to download Font Awesome.
3. Unzip the downloaded file, then locate the **otfs** folder.
4. Rename the **Font Awesome 5 Free-Solid-900.otf** file to **FontAwesome.otf** (you can keep the original name, but it's just less to type if you rename it). Your filename may be different since Font Awesome is continually updating but it should be similar.
5. Copy **FontAwesome.otf** to the **Resources/Fonts** folder in the **News** project.

Alright – now, we need to register Font Awesome with .NET MAUI.

Configuring .NET MAUI to use Font Awesome

It would be nice if all we needed to do was to copy the font file into the project folders. A lot does happen with just that action. The default .NET MAUI template includes all the fonts in the **Resources/Fonts** folder with the following item definition in the **News.csproj** file:

```
<!-- Custom Fonts -->
<MauiFont Include="Resources\Fonts\*" />
```

This ensures that the font files are processed and included in the app package automatically. What is left is to register the font with the .NET MAUI runtime so that it is available to our XAML resources. To do that, add the following highlighted line to the **MaiProgram.cs** file:

```
.ConfigureFonts(fonts =>
{
    fonts.AddFont("FontAwesome.otf", "FontAwesome");
    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
    fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
})
```

This line adds an alias that we can use in the next section to create static resources. The first parameter is the filename for the font file, while the second is the alias for the font that we can use in the **FontFamily** attribute.

All that is left is to define some icons in a resource dictionary.

Defining some icons in the resource dictionary

Now that we have defined the font, we will put that to use and define five icons to use in our app. We'll add the XAML first; then, we'll examine one of the **FontImage** tags.

Proceed as follows:

1. In the **News** project, open **App.xaml**.
2. Add the following code marked in bold under the existing **ResourceDictionary.MergedDictionaries** tag:

```
<ResourceDictionary>
  <ResourceDictionary.MergedDictionaries>
    <ResourceDictionary Source="Resources/Styles/Colors.xaml" />
    <ResourceDictionary Source="Resources/Styles/Styles.xaml" />
  </ResourceDictionary.MergedDictionaries>
  <FontImage x:Key="HomeIcon" FontFamily="FontAwesome" Glyph="&#xf015;" Size="22" Color="Black" />
  <FontImage x:Key="HeadlinesIcon" FontFamily="FontAwesome" Glyph="&#xf70e;" Size="22" />
  <FontImage x:Key="NewsIcon" FontFamily="FontAwesome" Glyph="&#xf1ea;" Size="22" />
  <FontImage x:Key="SettingsIcon" FontFamily="FontAwesome" Glyph="&#xf013;" Size="22" Color="Black" />
  <FontImage x:Key="AboutIcon" FontFamily="FontAwesome" Glyph="&#xf05a;" Size="22" Color="Black" />
</ResourceDictionary>
```

FontImage is a class that can be used anywhere in .NET MAUI that expects an **ImageSource** object. It's designed to render one character (or glyph) into an image. The **FontImage** class needs a few attributes to work, detailed as follows:

- A key for reference that will be used by other views in the application.
- A **FontFamily** resource that uses the alias reference back to the **Font** resource that we defined in the preceding section.
- The **Glyph** object, which represents the image to be shown. To find which image these cryptic values refer to, check out fontawesome.com, click on **Icons**, select **free and open-source icons**, and start browsing.
- **Size** and **Color**. These are not strictly needed but they're nice to define. They are used for a few of the icons in this app so that they render properly for light themes.

Font Awesome is now installed and configured. We've done a lot of work to get to the actual topic of this chapter. It's time to define the shell!

Defining the shell

As stated earlier, .NET MAUI Shell is the newest way of defining the structure of your app. The different projects in this book use alternate ways of defining the overall structure of the app, but .NET MAUI Shell is, in our opinion, the best way of defining the UI structure. We hope you find it as exciting as we do!

Defining the basic structure

We're going to start by defining a basic structure for the app without really adding any of our defined views to it. After that, we'll add the actual views one by one. But let's start by adding some content and creating **ContentPage** objects using XAML directly. Follow these two steps to do so:

1. In the **News** project, open the **AppShell.xaml** file.
2. Alter the file so that it looks like the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:local="clr-namespace:News">
```

```

x:Class="News.AppShell">
<FlyoutItem Title="Home" Icon="{StaticResource HomeIcon}">
  <ShellContent Title="Headlines" Icon="{StaticResource HeadlinesIcon}" >
    <ContentPage Title="Headlines" />
  </ShellContent>
  <Tab Title="News" Icon="{StaticResource NewsIcon}">
    <ContentPage Title="Local" />
    <ContentPage Title="Global" />
  </Tab>
</FlyoutItem>
<FlyoutItem Title="Settings" Icon="{StaticResource SettingsIcon}">
  <ContentPage Title="Settings" />
</FlyoutItem>
<ShellContent Title="About" Icon="{StaticResource AboutIcon}">
  <ContentPage Title="About"/>
</ShellContent>
</Shell>

```

Let's break this down. First, by default, the .NET MAUI Shell template disables the flyout menu. Since we want to use it in this app, you must remove the line that disables it. The direct children of Shell itself are two **FlyoutItem** objects and one **ShellContent** object. All three of these have **Title** and **Icon** properties defined, as shown in the following screenshot. The icons are referenced to the Font Awesome resources we created earlier. This will render a flyout, as shown in the following screenshot:

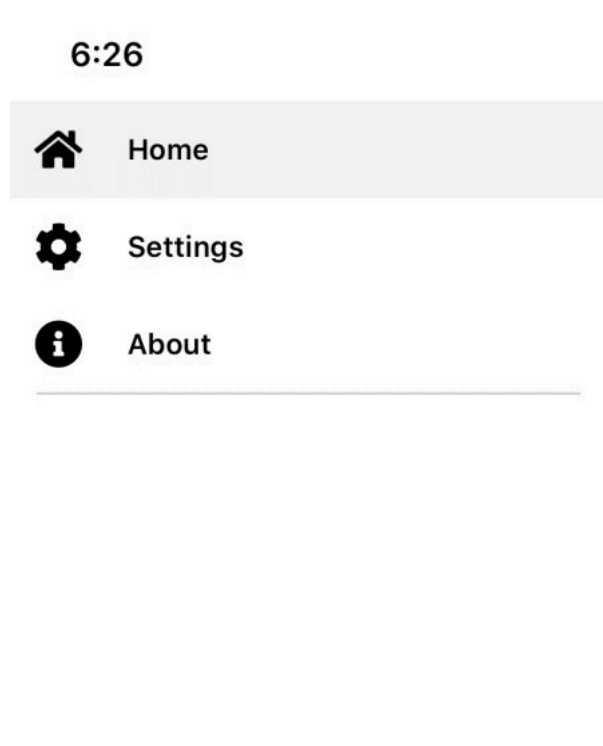


Figure 4.7 – The app flyout

The flyout is accessible by swiping in from the left. **Flyout** objects can have multiple children, while the **ShellContent** element can only have a

single child.

The **Home** flyout item is the most complex example. It has two children; **ShellContent** with a page with the title **Headlines** and a tab that defines two child pages on its own. The first level of children will render the tab bar at the bottom of the app, as shown in the following screenshot. The second level of children, under the **Tab** element with the **News** title, will be rendered as a tab bar directly under the title of the navigation bar at the top:

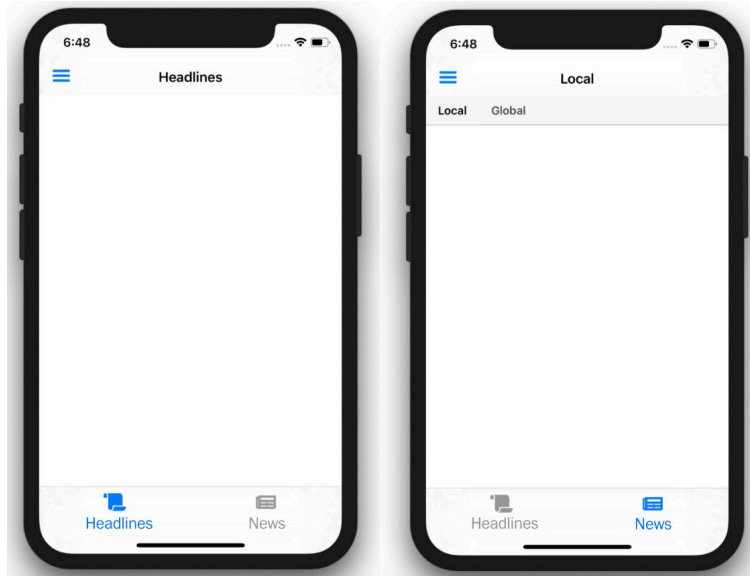


Figure 4.8 – The Shell tabs and pages

The **Settings** and **About** flyouts will simply render the pages that they define.

Making the app run

It's time to try the app out and see whether it looks like the screenshots shown throughout this chapter. The app should now run. If not, stay calm and simply go through the code again. Once you are done navigating around the app, we can create a news service to fetch news from, and extend all those views that we've created.

Creating the news service

To find cool content, we'll consume an already existing **News** API provided by newsapi.org. To do so, we must register for an API key that we can use to request the news. If you aren't comfortable with doing so, you can mock the news service if you would like, instead of using the API.

The first thing we must do is obtain an API key.

Obtaining an API key

The process of registering is pretty straightforward. Be aware, however, that the UI of newsapi.org might have changed by the time you read this.

OK – let's get that key:

1. Browse to <https://newsapi.org/>.
2. Click **Get API key**.
3. Fill out the form, as illustrated in the following screenshot:

Register for API key

First name

Email address

Choose a password

You are...

☒ I am an individual ☐ I am a business or am working on behalf of a business 

Jag är inte en robot

☒ I agree to the [terms](#).☒ I promise to add an [attribution link](#) on my website or app to NewsAPI.org.

Figure 4.9 – Register for API key

4. Copy the API key provided on the next page, as illustrated in the following screenshot:

Registration complete

Your API key is: 9d7c39bb[REDACTED]a3bf7ebbec5f7

For help getting started please look at our [getting started guide](#).

We post API status updates and other news on our Twitter feed, so please follow us there if that's important to you:

[Follow @NewsAPIorg](#)[My account](#)

Figure 4.10 – Registration complete

Now, we need a place to store the key for easy access. We'll create a static class that will hold the key for us. Proceed as follows:

1. In the **News** project, create a new class called **Settings** in the root folder.

2. Add the code shown in the following snippet, replacing the placeholder text with the API key you obtained in the preceding steps:

```
namespace News;
internal static class Settings
{
    public static string NewsApiKey => "<Your APIKEY Here>";
}
```

The important thing here is that you copy and paste the key into the file. Now, we need models.

A NOTE ON TOKENS AND OTHER SECRETS

*This is not a recommended way to store an API key or any other token that should be secured in your app. To securely store tokens and other data, you should use **secure storage** (see <https://learn.microsoft.com/en-us/dotnet/maui/platform-integration/storage/secure-storage>) and obtain the data from a secure server, preferably gated by some form of user authentication. You could also require the user to provide the API key through a settings page – hint, hint.*

Creating the models

The data returned from the API needs to go somewhere, and the most convenient way of accessing it would be to deserialize the data into **Plain Old CLR Objects (POCO)**, also known as regular C# classes). We usually call these POCO objects models, and they usually like to live in a folder called **Models**. Let's create our models:

1. In the **News** project, create a new folder called **Models**.
2. In the **Models** folder, add a new class called **NewsApiModels**.
3. Add the following code to the class:

```
namespace News.Models;
using System;
using System.Collections.Generic;
using System.Text.Json.Serialization;
public class Source
{
    [JsonPropertyName("id")]
    public string Id { get; set; }
    [JsonPropertyName("name")]
    public string Name { get; set; }
}
public class Article
{
    [JsonPropertyName("source")]
    public Source Source { get; set; }
    [JsonPropertyName("author")]
    public string Author { get; set; }
    [JsonPropertyName("title")]
    public string Title { get; set; }
    [JsonPropertyName("description")]
    public string Description { get; set; }
    [JsonPropertyName("url")]
```



```

public string Url { get; set; }
[JsonPropertyName("urlToImage")]
public string UrlToImage { get; set; }
[JsonPropertyName("publishedAt")]
public DateTime PublishedAt { get; set; }
[JsonPropertyName("content")]
public string Content { get; set; }
}
public class NewsResult
{
    [JsonPropertyName("status")]
    public string Status { get; set; }
    [JsonPropertyName("totalResults")]
    public int TotalResults { get; set; }
    [JsonPropertyName("articles")]
    public List<Article> Articles { get; set; }
}

```

The **JsonPropertyName** attributes on each property allow the **System.Text.Json** deserializer to map the name from the JSON received from the web API into the C# object. When we call the API, the API will return a **NewsResult** object that will, in turn, contain a list of articles. The next step is to create a service that will wrap the API and allow us to access the latest news.

TIP FOR CREATING POCO CLASSES

*If you ever need to create a class model out of a bunch of **JavaScript Object Notation (JSON)**, you can use the **Paste JSON as Classes** tool in Visual Studio for Windows (**Edit** | **Paste Special** | **Paste JSON as Classes**).*

Creating a service class

The service class will wrap the API so that we can access it in a nice .NET-ish kind of way.

But we'll start by defining an enum that will define the scope of the news that we request.

Creating the NewsScope enum

The **NewsScope** enum defines the different kinds of news that our service supports. Let's add it by following these few steps:

1. In the **News** project, create a new folder called **Services**.
2. In the **Services** folder, add a new file called **NewsScope.cs**.
3. Add the following code to that file:

```

namespace News.Services;
public enum NewsScope
{
    Headlines,
    Local,
    Global
}

```

The next step is to create the **NewsService** class that will wrap the call to the **News API**.

Creating the **NewsService** class

The purpose of the **NewsService** class is to wrap the HTTP calls to the news REST API and make them easily accessible to our code in the form of regular .NET method calls. To make it easier to replace the source of the news – for example, to use a mock in tests – we will use an interface.

To create the **INewsService** interface, follow these steps:

1. In the **Services** folder, create a new interface called **INewsService**.
2. Edit the interface so that it looks like this:

```
namespace News.Services;
using News.Models;
public interface INewsService
{
    public Task<NewsResult> GetNews(NewsScope scope);
}
```

Creating the **NewsService** class is now quite straightforward. Follow these steps:

1. In the **Services** folder, create a new class called **NewsService**.
2. Edit the class so that it looks like this:

```
namespace News.Services;
using News.Models;
using System.Net.Http.Json;
public class NewsService : INewsService, IDisposable
{
    private bool disposedValue;
    const string UriBase = "https://newsapi.org/v2";
    readonly HttpClient httpClient = new() {
        BaseAddress = new(UriBase),
        DefaultRequestHeaders = { { "user-agent", "maui-projects-news/1.0" } }
    };
    public async Task<NewsResult> GetNews(NewsScope scope)
    {
        NewsResult result;
        string url = GetUrl(scope);
        try
        {
            result = await httpClient.GetFromJsonAsync<NewsResult>(url);
        }
        catch (Exception ex) {
            result = new() { Articles = new() { new() { Title = $"HTTP Get failed: {ex.Message}" } }
        }
        return result;
    }
    private string GetUrl(NewsScope scope) => scope switch
    {
        NewsScope.Headlines => Headlines,
        NewsScope.Global => Global,
        NewsScope.Local => Local,
    }
}
```

```

        _ => throw new Exception("Undefined scope")
    };
    private static string Headlines => $"{UriBase}/top-headlines?country=us&apiKey={Settings.Ne
    private static string Local => $"{UriBase}/everything?q=local&apiKey={Settings.NewsApiKey}"
    private static string Global => $"{UriBase}/everything?q=global&apiKey={Settings.NewsApiKey}
    protected virtual void Dispose(bool disposing)
    {
        if (!disposedValue)
        {
            if (disposing)
            {
                httpClient.Dispose();
            }
            disposedValue = true;
        }
    }
    public void Dispose()
    {
        // Do not change this code. Put cleanup code in 'Dispose(bool disposing)' method
        Dispose(disposing: true);
        GC.SuppressFinalize(this);
    }
}

```

The **NewsService** class is composed of five methods; I know there are technically eight, but we will get to that in a moment.

The first method, **GetNews**, is the method that we will eventually call from our app. It takes one parameter, **scope**, which is the enum that we created earlier. Depending on the value of this parameter, we will get different types of news. The first thing this method does is resolve the URL to call, and it does so by calling the **GetUrl** method with the scope.

The **GetUrl** method uses a switch expression to resolve the URL and, depending on the value of the scope parameter passed, returns one of three URLs. The URL points to the REST API of the **News** API with some predefined query parameters and the API key that we registered for.

When we've resolved the correct URL, we are ready to make the HTTP request and download the news in the form of JSON. The built-in **HttpClient** class in .NET does a fine job of fetching the JSON for us. All that is left after acquiring the data is to deserialize it into the news models that we defined earlier.

Let's say a word or two about the remainder of the methods and the **HttpClient** class. **HttpClient** is now the recommended class to use when requesting data from the web. It is a much safer implementation than was previously available. It has been shipping with .NET 5+ and is available as a separate NuGet package for previous versions. With that, there are a few peculiarities when using **HttpClient**.

First, **HttpClient** can hold onto native resources, so it must be properly disposed of. To properly dispose of **HttpClient**, we need to derive from and implement **IDisposable**. This is the reason for the extra methods that

are in the class, **Dispose(bool)** and **Dispose()**. All they do is make sure that the instance of **HttpClient** is disposed of properly.

Second, **HttpClient** will pool those native resources, so it is recommended that you reuse an instance of **HttpClient** as much as possible. This is the reason for creating the **HttpClient** instance in the **NewsService** constructor.

Final word – since the **GetFromJsonAsync** call can throw an exception and it is called from an **async** method, you must handle the exception; otherwise, it will be lost on the executing thread and your only indication that something is wrong will be the fact that you have no items. For this app, we are just going to create a **NewsResult** object that contains one **Article** that has the exception in it so that something is displayed. There are much better ways of handling errors, but this will do for this app.

The next step is to wire up the **NewsService** class.

Wiring up the NewsService class

We are now ready to wire up the **NewsService** class in our app and integrate it with a real news source. We will extend all the existing **ViewModels** that we have and define the UI elements to be able to render the news in the **Views**.

Extending the HeadlinesViewModel class

In MVVM, **ViewModel** is the place to handle the logic of the app. The model is the news data that we will get from our **NewsService** class. We will now extend the **HeadlinesViewModel** class so that it uses **NewsService** to fetch news:

1. In the **News** project, expand the **ViewModels** folder and open the **HeadlinesViewModel.cs** file.
2. Add the following code marked in bold and resolve the references:

```
namespace News.ViewModels;
using System.Threading.Tasks;
using System.Web;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using News.Models;
using News.Services;
public partial class HeadlinesViewModel : ViewModel
{
    private readonly INewsService newsService;
    [ObservableProperty]
    private NewsResult currentNews;
    public HeadlinesViewModel(INewsService newsService)
    {
        this.newsService = newsService;
    }
    public async Task Initialize(string scope) =>
        await Initialize(scope.ToLower()) switch
        {
```

```

        "local" => NewsScope.Local,
        "global" => NewsScope.Global,
        "headlines" => NewsScope.Headlines,
        _ => NewsScope.Headlines
    });
    public async Task Initialize(NewsScope scope)
    {
        CurrentNews = await newsService.GetNews(scope);
    }
    [RelayCommand]
    public void ItemSelected(object selectedItem)
    {
        var selectedArticle = selectedItem as Article;
        var url = HttpUtility.UrlEncode(selectedArticle.Url);
        // Placeholder for more code later on
    }
}

```

Since we are using (constructor) dependency injection, we need to inject our dependencies into the constructor. The only dependency this **ViewModel** has is **NewsService**, and we store it internally in the class as a field.

The **CurrentNews** property is defined to get something to bind the UI to.

Then, we have two **Initialize** methods – one that takes **scope** as an enum and one that takes **scope** as a string. The string overload of the **Initialize** method will be used in XAML. It simply translates the string into the enum representation of **scope** and then calls the other **Initialize** method, which, in turn, calls the **GetNews(...)** method on the news service.

The property at the end, **ItemSelected**, returns a .NET MAUI command that we will wire up to respond to when the user of the app selects an item. Half of the method is implemented from the start. The selected item will be passed into the method. Then, we encode the URL to the article since we will be passing it as a query parameter when we navigate within the app. We will get back to the navigation part in a while.

If you are curious about the **ObservableProperty** and **RelayCommand** attributes, refresh your memory by reviewing *Chapter 2, Building Our First .NET MAUI App*.

Now that we have the code to fetch the data, we'll move on to defining the UI to display it.

Extending HeadlinesView

HeadlinesView is a shared view that will be used in several places in the app. The purpose of this view is to display a list of articles and to allow for navigation from one article into a web browser that will display the entire article.

To extend **HeadlinesView** we must do two things – first, must we edit the XAML and define the UI; then, we need to add some code to initialize it.

Proceed as follows:

1. In the **News** project, expand the **Views** folder and open the **HeadlinesView.xaml** file.
2. Edit the XAML, as shown in the following code block:

```
<?xml version="1.0" encoding="UTF-8"?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:views="clr-namespace:News.Views"
              xmlns:viewModels="clr-namespace:News.ViewModels"
              xmlns:models="clr-namespace:News.Models"
              x:Name="headlinesview"
              x:Class="News.Views.HeadlinesView"
              x:DataType="viewModels:HeadlinesViewModel"
              Title="Home" Padding="14">
  <CollectionView ItemsSource="{Binding CurrentNews.Articles}">
    <CollectionView.EmptyView>
      <Label Text="Loading" />
    </CollectionView.EmptyView>
    <CollectionView.ItemTemplate>
      <DataTemplate x:DataType="models:Article">
        <ContentView>
          <ContentView.GestureRecognizers>
            <TapGestureRecognizer Command="{Binding BindingContext.ItemSelectedCommand, Source="
          </ContentView.GestureRecognizers>
          <views:ArticleItem />
        </ContentView>
      </DataTemplate>
    </CollectionView.ItemTemplate>
  </CollectionView>
</ContentPage>
```

HeadlinesView uses **CollectionView** to display the list of articles. The **ItemsSource** property is set to the **CurrentNews.Articles** property of **ViewModel**, which, after loading news, should contain a list of news. While the list is empty or loading, we display a loading label, defined within the **CollectionView.EmptyView** element. You could, of course, create any valid UI inside of that tag to create a cooler loading screen.

Each article in the **CurrentNews.Articles** list will be rendered using whatever is inside the **CollectionView.ItemTemplate** element, and it's what's inside of the **ContentView** element that will represent the actual item. The article will be rendered using an **ArticleItem** view, which is a custom control that we defined earlier. We will define this view after we are done with this view.

To enable navigation from the view, we need to detect when a user clicks on a specific article. We can do so by adding **TapGestureRecognizer** and binding it to the **ItemSelectedCommand** property of the root **ViewModel**. The **Source={x:Reference headlinesview}}** snippet is what references the current context back to the root of the page, not the current article in the list that we are iterating. If we didn't specify the source, the binding engine would try to bind the **ItemSelectedCommand** property to a property defined in the current article of the **CurrentNews.Articles** property.

That's all for the GUI part. Now, we need to alter the code-behind to enable initialization based on data that we will pass from the XAML itself. Follow these steps to make it happen:

1. In the **News** project, open the **HeadlinesView.xaml.cs** code-behind file.
2. Add the following code marked in bold to the file:

```
namespace News.Views
using System.Threading.Tasks;
using News.Services;
using News.ViewModels;
public partial class HeadlinesView : ContentPage
{
    readonly HeadlinesViewModel viewModel;
    public HeadlinesView(HeadlinesViewModel viewModel)
    {
        this.viewModel = viewModel;
        InitializeComponent();
        Task.Run(async () => await Initialize(GetScopeFromRoute()));
    }
    private async Task Initialize(string scope)
    {
        BindingContext = viewModel;
        await viewModel.Initialize(scope);
    }
    private string GetScopeFromRoute()
    {
        var route = Shell.Current.CurrentState.Location
        .OriginalString.Split("/").LastOrDefault();
        return route;
    }
}
```

Usually, we don't want to add code to the code-behind of a view directly, but we need to make an exception so that arguments can be passed from XAML to our **ViewModel**.

Based on the routing information that was used to create the view, we will initialize **ViewModel** differently. The **GetScopeFromRoute** method will parse the location information from **Shell** to determine which scope is used to query the news service. Then, we can call a private method that creates an instance of **HeadlinesViewModel** for us, sets it as the binding context of the view, and calls the **Initialize()** method on **ViewModel**, which makes the REST call to the **News** API. We will define the routes when we edit the shell file.

But first, we need to extend **ContentView** of **ArticleItem** so that it displays a single-row item in the news lists.

Extending ContentView of ArticleItem

ContentView of **ArticleItem** represents one item in a list of news, as shown in the following screenshot:



NFL Week 17 bold predictions: DROY candidates Sauce Gardner, Tariq Woolen each record INT - NFL.com
December 30, 2022

Figure 4.11 – A sample news item

To create the layout shown in *Figure 4.11*, we will use a **Grid** control.

Follow these steps to create the layout:

1. In the **News** project, expand the **Views** folder and open the **ArticleItem.xaml** file.
2. Edit the XAML code, as shown in the following code block:

```
<?xml version="1.0" encoding="UTF-8"?>
<ContentView xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:models="clr-namespace:News.Models"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="News.Views.ArticleItem"
    x:DataType="models:Article">
    <Grid Margin="0">
        <Grid.RowDefinitions>
            <RowDefinition Height="10" />
            <RowDefinition Height="40" />
            <RowDefinition Height="15" />
            <RowDefinition Height="10" />
            <RowDefinition Height="1" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="65" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>
        <Label Grid.Row="1" Grid.Column="1" Text="{Binding Title}" Padding="10,0" FontSize="Small"
        <Label Grid.Row="2" Grid.Column="1" Text="{Binding PublishedAt, StringFormat='{0:MMMM d, yy"
        <Border Grid.Row="1" Grid.RowSpan="2" StrokeShape="RoundRectangle 15,15,15,15" Padding="0"
            <Image Source="{Binding UrlToImage}" Aspect="AspectFill" HeightRequest="55" HorizontalOpt
        </Border>
        <BoxView Grid.Row="4" Grid.ColumnSpan="2" BackgroundColor="LightGray" />
    </Grid>
</ContentView>
```

The preceding XAML code defines a grid layout with two columns and five rows. The **Grid.Row** and **Grid.Column** attributes position child elements into the grid, while the **Grid.ColumnSpan** attribute allows for a control to span multiple columns.

A rounded image can be achieved using a **Border** element with **StrokeShape="RoundRectangle 15,15,15,15"** specified in conjunction with the **Image Aspect** attribute set to **AspectFill**.

Strings in labels can be formatted directly in the binding statement. Check out the **Text="{Binding PublishedAt, StringFormat='{0:MMMM d, yyyy}'}"** line of code, which formats a date into a specific string format.

Lastly, the gray divider line is **BoxView** at the end of the XAML code.

Now that we have created **NewsService** and fixed all its related views, it's time to make use of them.

Adding to dependency injection

Since **HeadlinesViewModel** depends on **INewsService**, we need to register it in our dependency injection container (please see the *Wiring up dependency injection* section in *Chapter 2* for more details on dependency injection in .NET MAUI). Follow these steps to do so:

1. In the **News** project, open the **MauiProgram.cs** file.
2. Locate the **RegisterAppTypes()** method and add the following line marked in bold:

```
public static MauiAppBuilder RegisterAppTypes(this MauiAppBuilder mauiAppBuilder)
{
    // Services
    mauiAppBuilder.Services.AddSingleton<Services.INewsService>((serviceProvider) => new Servi
    // ViewModels
    mauiAppBuilder.Services.AddTransient<ViewModels.HeadlinesViewModel>();
    //Views
    mauiAppBuilder.Services.AddTransient<Views.AboutView>();
    mauiAppBuilder.Services.AddTransient<Views.ArticleView>();
    mauiAppBuilder.Services.AddTransient<Views.HeadlinesView>();
    return mauiAppBuilder;
}
```

This will allow for the dependency injection of **NewsService**.

Adding a ContentTemplate attribute

So far, we only have placeholder code in our **AppShell** file. Let's replace this with actual content, as follows:

1. In the **News** project, open **AppShell.xaml**.
2. Locate the **FlyoutItem** element with the title set to **Home**.
3. Edit the XAML so that the **ShellContent** element becomes self-closing, add the following **ContentTemplate** attribute marked in bold, and replace the contents of the **Tab** element:

```
<Shell
    x:Class="News.AppShell"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:views="clr-namespace:News.Views"
    xmlns:local="clr-namespace:News">
    <FlyoutItem Title="Home" Icon="{StaticResource HomeIcon}">
        <ShellContent Title="Headlines" Route="headlines" Icon="{StaticResource HeadlinesIcon}"
        <Tab Title="News" Route="news" Icon="{StaticResource NewsIcon}">
            <ShellContent Title="Local" Route="local" ContentTemplate="{DataTemplate views:Head
            <ShellContent Title="Global" Route="global" ContentTemplate="{DataTemplate views:He
        </Tab>
    </FlyoutItem>
    <FlyoutItem Title="Settings" Icon="{StaticResource SettingsIcon}">
        <ContentPage Title="Settings" />
    </FlyoutItem>
```

```
<ShellContent Title="About" Icon="{StaticResource AboutIcon}">
  <ContentPage Title="About"/>
</ShellContent>
</Shell>
```

Two things are going on here. The first is that we specify the content of **ShellContent** using the **ContentTemplate** attribute. This means that we point to the type of view that we want to have created when the shell becomes visible. Usually, you want to defer the creation of a view until right before it is going to be displayed, and the **ContentTemplate** attribute gives you that. Notice that the **Route** attribute for that **FlyoutItem** is set to **headlines**.

The second thing is that we have the same thing going on below for the **Local** and **Global** news but instead using the **local** and **global** routes.

If you run the app at this point, you should end up with something that looks like the following screenshot:

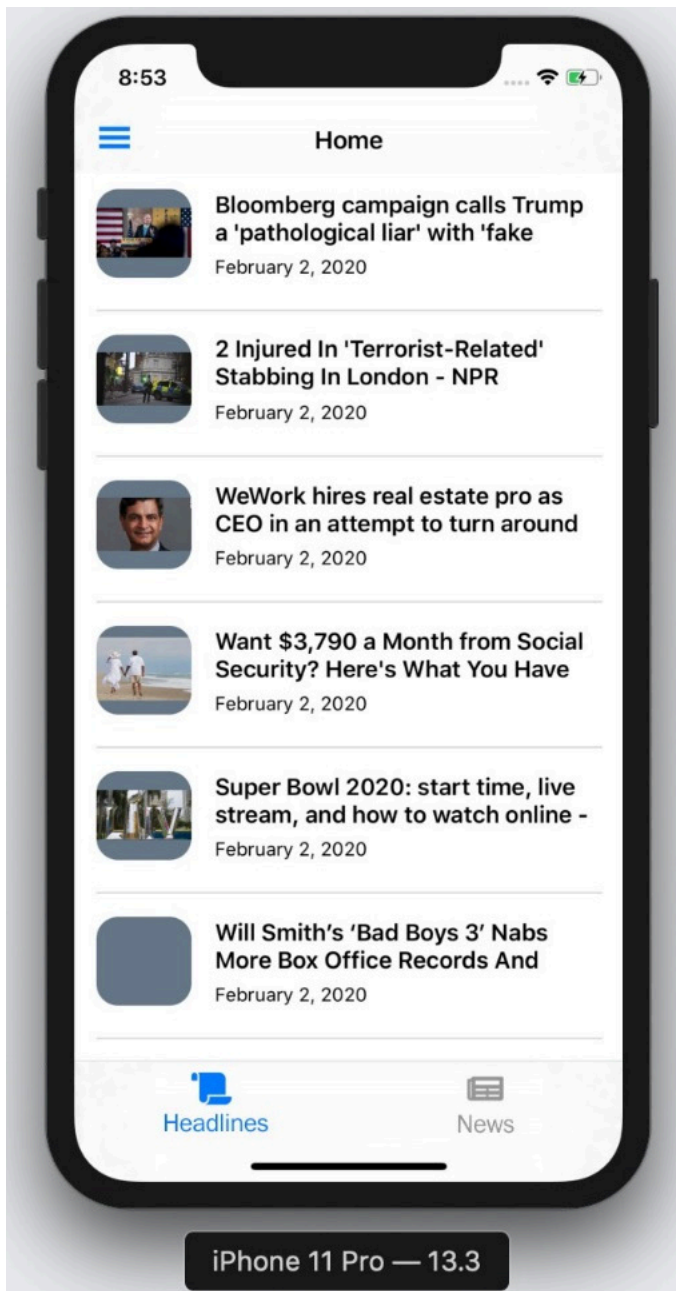


Figure 4.12 – The main list view

The last thing we need to implement is how we can view the articles when we tap on the item in the list.

Handling navigation

We are on the home stretch now for this app. The only thing we need to do is implement navigation to the article view, which will display the entire article in a web view. Since we are using Shell, we will be navigating using routes. Routes can be registered directly in the Shell markup – for example, in the `AppShell1.xaml` file. We can do this by using `Route` attributes on the `ShellContent` elements, as we did in the previous section.

In the following code, we will add a route programmatically and register a view to handle it for us. We will also create a navigation service to abstract the concept of navigation a bit.

So, buckle up, and let's finish this app!

Creating the navigation service

The first step is to define an interface that will wrap .NET MAUI navigation. Why do we do this? Well, it's a good practice to separate the interfaces from the implementation; it makes unit testing easier, and so on.

Creating the `INavigation` interface

The `INavigation` interface is simple, and we will overshoot the target a little bit. We are only interested in the `NavigateTo` method, but we will add the `PushModal()` and `PopModal()` methods since it is likely that you will use them if you continue extending the app.

Adding the navigation interface is simple, as the following steps illustrate:

1. In the **News** project, expand the **ViewModels** folder and add a new file called **INavigation.cs**.
2. Add the following code to the file:

```
namespace News.ViewModels;
public interface INavigate
{
    Task NavigateTo(string route);
    Task PushModal(Page page);
    Task PopModal();
}
```

The `NavigateTo()` method declaration takes the route we want to navigate to. This is the method that we will be calling. The `PushModal()` method adds a new page on top of the navigation stack as a modal page, forcing the user only to interact with this specific page. The `PopModal()` method removes it from the navigation stack. So, if you use the `PushModal()` method, make sure that you give the user a way to pop it off the stack.

Otherwise, you will be stuck viewing the modal page forever.

That's all for the interface. Let's create an implementation using .NET MAUI Shell.

Implementing the `INavigation` interface using .NET MAUI Shell

The implementation is very straightforward since each method is only calling .NET MAUI static methods provided by the Shell API.

Create the **Navigator** class, as shown in the following steps:

1. In the **News** project, add a new class called **Navigator**.
2. Add the following code to the class:

```
namespace News;
```

```
using News.ViewModels;
public class Navigator : INavigate
{
    public async Task NavigateTo(string route) => await Shell.Current.GoToAsync(route);
    public async Task PushModal(Page page) => await Shell.Current.Navigation.PushModalAsync(page);
    public async Task PopModal() => await Shell.Current.Navigation.PopModalAsync();
}
```

This is simply pass-through code that calls already existing methods. Now, we need to register the type with our dependency injection container so that it can be consumed by the **ViewModel** class.

Registering the Navigator class with dependency injection

For the **ViewModel** class and its descendants to have access to the **Navigator** instance, we will have to register it with the container, just like we did with **NewsService**; just follow these steps:

1. In the **News** project, open the **MauiProgram.cs** file.
2. Find the **RegisterAppTypes** method and add the following highlighted code:

```
public static MauiAppBuilder RegisterAppTypes(this MauiAppBuilder mauiAppBuilder)
{
    // Services
    mauiAppBuilder.Services.AddSingleton<Services.INewsService>((serviceProvider) => new NewsService(serviceProvider));
    mauiAppBuilder.Services.AddSingleton<ViewModels.INavigate>((serviceProvider) => new Navigator(serviceProvider));
    // ViewModels
    ...
}
```

Now, we can add the **INavigate** interface to the **ViewModel** class and its descendants.

Adding the INavigate interface to the ViewModel class

To be able to access **Navigator**, we must extend the **ViewModel** base class, making it available to all **ViewModels**. Proceed as follows:

1. In the **News** project, open the **ViewModels** folder, and then open the **ViewModel.cs** file.
2. Add the following highlighted code to the class:

```
public abstract class ViewModel
{
    public INavigate Navigation { get; init; }
    internal ViewModel(INavigate navigation) => Navigation = navigation;
}
```

3. Open the **HeadlinesViewModel.cs** file and make the highlighted changes to the constructor:

```
public HeadlinesViewModel(INewsService newsService, INavigate navigation) : base(navigation)
```

The base **ViewModel** now exposes the **Navigator** property through the **INavigate** interface. At this point, we are ready to wire up the navigation to our **Article** view.

Navigating using routes

Routes are a very handy way to navigate since they abstract the creation of the page away. All we need to know is the route to the view we want to navigate to – .NET MAUI Shell takes care of the rest for us. If you are familiar with how web navigation works, you might recognize the way that we pass arguments in routes. They are passed as query parameters.

Finalizing the ItemSelected command

Previously, we defined the **ItemSelected** method in the **HeadlinesViewModel** class. Now, it's time to add code that will perform the navigation to **ArticleView**:

1. In the **News** project, expand the **ViewModels** folder and open **HeadlinesViewModel.cs**.
2. Locate the **ItemSelected** method and add the following line marked in bold:

```
[RelayCommand]
public async Task ItemSelected(object selectedItem)
{
    var selectedArticle = selectedItem as Article;
    var url = HttpUtility.UrlEncode(selectedArticle.Url);
    await Navigation.NavigateTo($"articleview?url={url}");
}
```

Here, we define a route called **articleview** that takes a query-line parameter called **url** that points to the URL of the article itself. It will look something like this: **articleview?url=www.mypage.com**. Only the data passed after the **url=** argument must be **URL encoded**, meaning that we need to encode it to replace characters that would interfere with the route itself. The URL encoding is done through the **HttpUtility.UrlEncode()** method, which is defined in **System.Web** for us.

The preceding **NavigateTo()** method call uses this encoded data in the query parameter. On the recipient side of the navigation call, we need to handle the incoming **url** parameter.

Extending ArticleView so that it receives query data

ArticleView is responsible for rendering the article for us. To keep things simple (and also illustrate that you don't always need **ViewModel**), we will not define **ViewModel** for this class; instead, we will define **BindingContext** as an instance of the **UrlWebViewSource** class.

Add the following code to the **ArticleView.xaml.cs** file:

1. In the **News** project, expand the **Views** folder and open the **ArticleView.xaml.cs** file.
2. Add the following code marked in bold to the file:

```
namespace News.Views;
using System.Web;
[QueryProperty("Url", "url")]
public partial class ArticleView : ContentPage
{
    public string Url
    {
        set
        {
            BindingContext = new UrlWebViewSource
            {
                Url = HttpUtility.UrlDecode(value)
            };
        }
    }
    public ArticleView()
    {
        InitializeComponent();
    }
}
```

ArticleView is dependent on a URL being set, and we do so by defining a set-only property called **Url**. When this property is set, it creates a new instance of **UrlWebViewSource** with the value of the property and assigns it to the page's **BindingContext**. This setter is called by the Shell framework since we added an attribute called **QueryProperty** to the class itself. It takes two arguments – the first is which property to set, while the second is the name of the **url** query parameter.

Since the data comes URL encoded, we need to decode it using the **HttpUtility.UrlDecode()** method.

With that, we have a binding context that points to the web page we want to display. Now, we just need to define **WebView** in the XAML.

Extending ArticleView with WebView

This page only has one purpose, and that is to display the web page from the URL that we passed to it. Let's add a **WebView** control to the page, as follows:

1. In the **News** project, expand the **Views** folder and open **ArticleView.xaml**.
2. Add the following highlighted XAML:

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="News.Views.ArticleView"
             Title="ArticleView">
```

```
<WebView Source="{Binding .}" />
</ContentPage>
```

The **WebView** control will take up all available space in the view. The source will be set to `.`, meaning that it will be the same as the **BindingContext** property of **ViewModel**. In this case, the **BindingContext** property is a **UrlWebViewSource** instance, which is exactly what **WebView** needs to navigate and display the content.

We've only got one step left – our app needs to know about the **ArticleView** route and what to do with it.

Registering the route

As mentioned previously, routes can be added declaratively in the XAML (**Route="MyDucks"**) or code, as shown here:

1. In the **News** project, open the **AppShell.xaml.cs** file.
2. Add the following lines of code marked in bold:

```
namespace News;
public partial class AppShell : Shell
{
    public AppShell()
    {
        InitializeComponent();
        Routing.RegisterRoute("articleview", typeof(Views.ArticleView));
    }
}
```

The **RegisterRoute()** method takes two arguments – the first is the route we want to use, and it's the one that we specify in the **NavigateTo()** calls. The second is the type of page (view) that we want to create – in our case, we want to create **ArticleView**.

Cool! That wraps it up. The app should now run, and you should be able to navigate to the article from any of your **CollectionView**s. Good work!

Summary

In this chapter, we learned how to define a navigation structure using .NET MAUI Shell, how to navigate to views using routes, and how to pass arguments between views in the form of a query string. There is a lot more to Shell, but this should get you started and confident enough to start exploring the Shell APIs. Also, keep in mind that the Shell APIs are constantly evolving, so make sure you check out the latest features available.

We also learned how to create an API client for an arbitrary REST API, which always comes in handy since most of the apps you will write need to communicate with a server at some point. There is a very good chance that the server will expose its data and functionality through a REST API.

If you are interested in extending the app even further, try designing your own **About** page, or allow the **News** API key to be set through the settings.

The next project will be about creating a match-making app, and how to create a swiping-enabled yes/no image selector app using nothing but .NET MAUI to render and animate cross-platform UI controls.